CPTS 360 Systems Programming Midterm Interview Example Questions

1. Expression Builder to Target Value

Scenario: Given the digits 1,2,…,9 in order, insert +, -, or nothing (concatenation) between them to make an expression that evaluates to 100. Describe an algorithm (pseudo-code) to generate all valid expressions and count how many equal 100. How would you avoid recomputing expression values from scratch? What data structure would you use during recursion?

Core Concepts: Recursion

Expected Logic:

1. 9 digits, 3 options (+, −, or " "), the total number of combinations is 3^8 = 6,561.
2. Concatenation (joining 1 and 2 to make 12) must happen before addition and subtraction.
3. Depth-first approach, moving from left to right until the end of the string is reached.

2. Graph data structure

Scenario: Implement a data structure for graphs that allows modification (insertion, deletion). It should be possible to store values at edges and nodes. It might be easiest to use a dictionary of (node, edgelist) to do this.

Core Concept: Dictionaries

Expected Logic:

- Use a dictionary of node-values for adjacency.
- Maintain a node-value dictionary.
- Check whether nodes exist, ensure symmetry for undirected graphs
- Clean up all incident edges when deleting a node

3. The Wrapping Packet Reader

Scenario: A circular buffer unsigned char buffer[1024] stores variable-length packets.

- head points to the start of the next packet.
- The first byte at head is the length L of the payload that follows.
- The payload occupies the next L bytes.

Write pseudo-code to read this packet into a separate array dest. The tricky part: the packet might wrap around the end of the buffer (e.g., starts at index 1020, length is 10, so it wraps to index 0).

Core Concept: Modular arithmetic, Array bounds checking, Split memory copying.

Expected Logic:

- Read length L = buffer[head].
- Loop i from 0 to L-1.
- Calculate source index: curr = (head + 1 + i) % 1024.
- Copy dest[i] = buffer[curr].
- Update state.

4. The my_system() Implementation

Scenario: Standard C provides a system() function that executes a shell command. Write pseudo-code to implement a simplified version called my_system(char *command). It should create a child process to execute the command and wait for it to finish before returning. You don't need to parse the command arguments, just pretend you can pass command directly to an exec function.

Core Concept: fork, exec, wait, Process creation.

Expected Logic:

- Call fork().
- Child: Call execl (or similar). Handle failure (call exit).
- Parent: Call wait or waitpid to pause until the child is done.
- Error Handling: Check if fork returns < 0.

5. String Subsequence

Scenario: Given two strings, write a program that outputs the shortest sequence of character insertions and deletions that turn one string into the other.

Core Concept: longest common subsequence, backtracking

Expected Logic:

- Use a 2D table dp[i][j] to keep track of min i chars of A to transform into j chars of B.
- No op needed in chars match, else: choose between del (dp[i-1][j]+1), ins (dp[i][j-1]+1).
- Use bottom-up approach for convenience.
- Trace backwards using dp[len(A)][len(B)].

6. The In-Place String Reversal

Scenario: Write a pseudo-code function that reverses a string in-place (without allocating a second string buffer). The function receives a char *str. Use of strlen trivializes this problem, think of a solution that does not use strlen.

Core Concept: Pointers, dereferencing, swapping logic.

Expected Logic:

- Iterate to find the null terminator \0 to determine the end pointer.
- Use two pointers (start and end) or indices.
- Swap characters at start and end, moving pointers inward until they meet.

7. The Linked List Filter (Kernel List)

Scenario: You have a singly linked list of struct process where each node contains int pid, int status, and struct process *next. Write pseudo-code to traverse the list and remove *all* nodes where status is equal to 0 (Zombie processes). Be careful not to break the list or lose pointers.

Core Concept: Linked Lists, Pointer manipulation, Edge cases (head removal).

Expected Logic:

- Handle the case where the head itself needs removal.
- Maintain a prev pointer while traversing.

- Link prev->next to current->next when deleting.

8. The "Tail" Implementation
Scenario: We need to read the *last* 100 bytes of a very large file (2 GB). We cannot read the whole file into memory. Write pseudo-code using open, lseek, and read to extract these bytes into a buffer.
Core Concept: File I/O, File Descriptors, Seeking.
Expected Logic:

- open() the file.
- Use lseek with SEEK_END to go to FileSize - 100.
- Handle the edge case where the file is smaller than 100 bytes.
- read() into buffer.

9. The Safe Integer Multiplier
Scenario: In systems programming, integer overflows can lead to security vulnerabilities. Write a function bool safe_multiply(int a, int b, int *result) that attempts to multiply a and b. If the result would overflow a standard 32-bit signed integer, return FALSE and leave result untouched. Otherwise, store the product and return TRUE.
Core Concept: Overflow detection, Boundary checking.
Expected Logic:

- Check for 0 (trivial success).
- Check if a * b > MAX_INT or a * b < MIN_INT without actually causing the overflow.
- Standard check: if (a > MAX_INT / b).

10. The Recursive Directory Size
Scenario: Write pseudo-code for a function get_dir_size(path) that calculates the total size of a directory, including the sizes of all files inside its sub-directories. Assume you have helper functions is_directory(path), list_files(path), and get_file_size(path).
Core Concept: Recursion, Filesystem traversal.
Expected Logic:

- Initialize total = 0.
- Loop through items in the path.
- If item is file: total += get_file_size.
- If item is directory: total += get_dir_size (Recursive call).
- Return total.

11. The Zombie Harvester
Scenario: A parent process has spawned 5 child processes. The parent needs to wait for *all* of them to finish. However, the parent does not know the PIDs of the children (it didn't save them). Write pseudo-code using wait or waitpid to ensure all children are reaped and no zombies are left.
Core Concept: wait, Process states, Return values.

Expected Logic:

- Loop calling wait(NULL).
- Break the loop only when wait returns -1 (indicating no children left) and errno indicates ECHILD.

12. The Environment Parser

Scenario: In C, the environment variables are passed as an array of strings char **environ, where each string is in the format "KEY=VALUE". Write pseudo-code to find the value of "HOME". You must parse the strings manually; do not use getenv.

Core Concept: String parsing, Pointer arrays, String comparison.

Expected Logic:

- Iterate through the environ array until NULL.
- For each string, check if it starts with "HOME=".
- If it matches, return a pointer to the character immediately after the '='.

13. The Stack Growth Detective

Scenario: Different computer architectures manage the stack differently: some grow "up" (towards higher memory addresses), others grow "down" (towards lower memory addresses). Write a pseudo-code function determine_direction() that returns either UP or DOWN. You must determine this programmatically by inspecting the addresses of local variables.

Hint: You might need to use recursion or a helper function call to create a new stack frame.

Core Concept: Memory Layout, Stack Frames, Address Comparison.

Expected Logic:

- Define a local variable a in the main function.
- Call a helper function (or recurse once).
- Inside the helper, define a local variable b.
- Compare the address of b vs a.
- If &b < &a, the stack grows DOWN. If &b > &a, it grows UP.

14. The Process Ancestry Tracer

Scenario: You have access to a struct task_struct representing a process, which contains a pointer struct task_struct *parent. Write a pseudo-code function bool is_ancestor(task_struct *target, task_struct *current) that returns TRUE if target is a parent (or grandparent, great-grandparent, etc.) of current.

Core Concept: Linked List Traversal, Pointer Comparison.

Expected Logic:

- Loop while current is not NULL (or the init process).
- Inside loop: Check if (current == target) return TRUE.
- Advance: current = current->parent.
- If loop finishes without match, return FALSE.

15. The Memory Block Merger

Scenario: In a custom memory allocator, free memory blocks are stored in a sorted linked list. Each node has addr, size, and next. Write pseudo-code to traverse the list and merge any two adjacent blocks into one larger block. Two blocks are adjacent if block->addr + block->size == block->next->addr.

Core Concept: Pointer manipulation, List compaction.

Expected Logic:

- Iterate through list with pointer curr.
- Compare curr end address with curr->next start address.
- If adjacent: curr->size += curr->next->size, remove curr->next node (don't advance curr yet, need to check next one too).
- Else: advance curr.

16. Recursive File Search

Scenario: Write a pseudo-code function search_file(directory, filename) that recursively searches for a file named filename inside a directory tree. It should return the full path if found, or NULL if not found.

Core Concept: Recursion, Tree Search (DFS).

Expected Logic:

- Iterate through items in directory.
- If item name matches filename: return current_path.
- If item is subdirectory: result = search_file(subdir, filename).
- If result is not NULL: return result.
- Return NULL if loop finishes.

17. The Path Canonicalizer

Scenario: Write a pseudo-code function that takes a raw file path like "/usr/lib/../bin/./gcc" and resolves the .. (parent) and . (current) references to output the canonical path: "/usr/bin/gcc". You can use a stack or an array to help.

Core Concept: Stack data structure, String parsing.

Expected Logic:

- Split string by /.
- Iterate tokens.
- If ..: Pop from stack (if not empty).
- If .: Do nothing.
- Else: Push to stack.
- Join stack elements with /.

18. The "argv" Builder

Scenario: When the kernel executes a program, it needs to convert a single command string like "ls -la /home" into an array of strings argv[] (ending with NULL). Write pseudo-code to parse this string in-place (by replacing spaces with \0) and fill an array of pointers to point to the start of each word.

Core Concept: Pointers, String Mutation, Null termination.

Expected Logic:

- Loop through string, skip leading whitespace.
- Store address of start of word in argv[i].
- Find end of word (space or \0).
- Replace space with \0, repeat until end of string.
- Set last argv entry to NULL.

19. The In-Place Whitespace Trimmer

Scenario: Write a pseudo-code function trim(char *str) that removes all leading and trailing whitespace from a string **in-place**. You cannot allocate a new buffer. Example: " hello " becomes "hello".

Core Concept: Two-pointer technique, Memory shifting.

Expected Logic:

- Find the index of the first non-space char (start).
- Find the index of the last non-space char (end).
- Shift characters from start...end to index 0.
- Write \0 at the new length.

20. The Recursive PID Sum

Scenario: Using the same struct task_struct from Question 14 (which has a list of children), write a recursive function sum_pids(task_struct *p) that returns the sum of the PID of process p AND the PIDs of all its descendants (children, grandchildren, etc.).

Core Concept: Tree Traversal, Accumulation via Recursion.

Expected Logic:

- Initialize sum = p->pid.
- Iterate through p->children.
- For each child: sum += sum_pids(child).
- Return sum.

21. The Signal Broadcaster

Scenario: Write pseudo-code for a function broadcast_signal(process, signal_id) that sends a signal to a process and recursively to all its children. Assume you have a helper send_signal(pid, sig).

Core Concept: Recursion, System administration logic.

Expected Logic:

- Call send_signal(process->pid, signal_id).
- Iterate through process->children.
- Recursively call broadcast_signal(child, signal_id).

22. The Sparse Page Detector

Scenario: In virtual memory, we sometimes want to know if a 4KB page is completely empty (all zeros) to optimize storage. Write pseudo-code bool is_zero_page(void *page) that checks this efficiently. Hint: Checking byte-by-byte is slow; check 4 bytes or 8 bytes at a time.

Core Concept: Pointer casting, Optimization, Loop unrolling.

Expected Logic:

- Cast page to long * (or uint64_t *).
- Loop from 0 to 4096 / sizeof(long).
- If *ptr != 0 return FALSE.
- Return TRUE.

23. The Safe String Copy

Scenario: Standard strcpy is dangerous. Write safe_copy(char *dest, char *src, int max_len) that copies src to dest but ensures dest is always null-terminated and never overflows max_len. It should return the number of characters copied (excluding null).

Core Concept: Buffer safety, Boundary conditions.

Expected Logic:

- Loop i from 0 to max_len - 1.
- If src[i] == \0, break.
- dest[i] = src[i].
- Explicitly set dest[i] = \0.
- Return i.

24. The Double-Linked Scheduler Insert

Scenario: You have a doubly linked list representing a scheduler queue. Nodes are struct task. Write pseudo-code to insert a new task new_task immediately *after* a specific existing task current_task. Handle the case where current_task is the tail.

Core Concept: Doubly Linked Lists, Pointer rewiring.

Expected Logic:

- Set new_task->prev = current_task.
- Set new_task->next = current_task->next.
- If current_task->next is not NULL: current_task->next->prev = new_task.
- Set current_task->next = new_task.

25. The Directory Depth Finder
Scenario: Write a recursive function max_depth(path) that returns the maximum nesting level of directories starting from path. A directory with no subdirectories has depth 0.
Core Concept: DFS, Max-finding algorithm.
Expected Logic:

- current_max = 0.
- List directories in path.
- For each subdir: d = max_depth(subdir).
- current_max = max(current_max, d + 1).
- Return current_max.

26. The Environment Variable Counter
Scenario: Using the char **environ array (which ends with a NULL pointer), write pseudo-code to count how many environment variables are currently set.
Core Concept: Pointer array traversal.
Expected Logic:

- Initialize count = 0.
- Loop while environ[count] != NULL.
- Increment count.
- Return count.

27. The Safe "atoi" (String to Int)
Scenario: Write a function string_to_int(char *str, int *out) that converts a string of digits to an integer. It returns TRUE on success and FALSE if the string contains non-digits or is empty.
Core Concept: Character parsing, ASCII math.
Expected Logic:

- Initialize res = 0.
- Loop through chars.
- Check if char < '0' || char > '9', return FALSE.
- res = res * 10 + (char - '0').
- Assign *out = res.

28. The Linked List Reversal (Recursive)
Scenario: Write a recursive function reverse_list(node) that reverses a singly linked list and returns the new head. The recursive step is new_head = reverse_list(node->next). Then you need to rewire the pointers.
Core Concept: Recursion, Stack frames as memory.
Expected Logic:

- Base case: if node or node->next is NULL, return node.
- Recurse: new_head = reverse_list(node->next).

- Rewire: node->next->next = node.
- Break old link: node->next = NULL.
- Return new_head.

29. The Bad Block Skipper

Scenario: You are copying data from a flash storage device. The device map has an array bool bad_blocks[]. If bad_blocks[i] is true, block i is corrupted. Write pseudo-code to read N valid blocks into a buffer. You might have to check more than N physical blocks if you encounter bad ones.

Core Concept: Skipping logic, Index decoupling.

Expected Logic:

- valid_read_count = 0.
- physical_index = 0.
- Loop while valid_read_count < N.
- If !bad_blocks[physical_index]: Read block, increment valid_read_count.
- Always increment physical_index.

30. The Run-Length Decoder

Scenario: You receive a compressed string where characters are followed by their count (e.g., "A3B2" means "AAABB"). Write pseudo-code to decode this into a buffer. Assume single-digit counts for simplicity.

Core Concept: String parsing, Buffer writing loops.

Expected Logic:

- Loop through source string with index i.
- char c = src[i].
- count = src[i+1] - '0'.
- Inner loop k from 0 to count: write c to destination.
- Increment i by 2.

31. Recursive Dimension Reduction

Scenario: Given a set of N-dimensional rectangular boxes, write a program that computes the volume of their union. Start with 2D and work your way up.

Core Concept: Inclusion-exclusion.

Expected Logic:

- Maintain a sorted list of active intervals.
- Volume += (current union in (d−1) dimensions) × (delta in sweep dimension)
- 2D → sweep x, compute 1D interval union.
- ND → sweep one axis, reduce to (N-1)D.